



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python os Module for Automation in Robotics

Objective

After completing this lab, students will be able to:

- Understand the role of the operating system in hardware and robotics automation.
- Navigate and inspect file system directories using Python.
- Create, rename, and delete files and folders programmatically.
- Analyze file paths and extract file metadata.
- Develop automated scripts for organizing robotics project files and logs.

Background

Robotics and automation systems constantly generate data — sensor readings, logs, configuration files, and reports — that must be stored, organized, and managed on disk. While Object-Oriented Programming (OOP) helps organize application logic, it does not by itself manage files, directories, or system resources.

The Python **os** module acts as a bridge between high-level Python code and the underlying operating system. It allows a script to manage files, directories, environment variables, and system processes, which is essential before a program can move on to communicating with physical hardware such as sensors and motors (for example, via PySerial).

System Integration Stack

1. High-Level Python OOP Logic — class structures and object management.
2. The Bridge: Host Operating System (via **os** module) — manages files, environment variables, directories, and processes.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



-
3. Physical Hardware Systems (Sensors / Motors) — direct interaction via PySerial, PyArduino, or GPIO pins.
-

Why Use the os Module?

The `os` module is used because it allows automation scripts to manage the file system reliably. Common uses include:

- Automating repetitive tasks such as starting scripts or managing data pipelines
- Organizing complex project structures and creating directories for experiments
- Generating and storing robot log files to track system health and errors
- Storing sensor data (CSV, JSON) systematically for analysis
- Reading and writing configuration files to adapt robot behavior without changing code

Real-World Examples

- **Autonomous Vehicles:** writing continuous data streams from LiDAR and cameras to high-speed storage
 - **Industrial Robotic Arms:** logging joint temperature, pressure metrics, and trajectory errors
 - **AI Training Datasets:** generating training logs and managing large, dynamically growing datasets
-

What is an Operating System?

An operating system is the software that supports a computer's basic functions, such as scheduling tasks, executing applications, and controlling peripherals. Common systems include Windows, Linux (widely used in robotics, e.g., ROS), and macOS.

Resource Management Areas



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Resource	Description
Files	Organizing data on storage drives
Processes	Managing running programs and CPU time
Devices	Handling communication with hardware peripherals (sensors, actuators)

Importing the Module

Syntax

```
Python  
import os
```

Once imported, the module gives program-level access to process environments, configurations, and system endpoints.

Common Applications

- **Path Navigation:** traverse directory paths programmatically
- **File & Directory Control:** create, delete, and iterate files and folders
- **Environment Access:** read and modify host machine configuration variables
- **System Operations:** invoke command-line prompts from within scripts

Cross-Platform Advantage: The `os` module abstracts platform differences, allowing path logic to work natively across Windows, macOS, and Linux without manual string modifications.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Core os Module Functions

1. Getting the Current Directory — `os.getcwd()`

Returns the current working directory of the running process.

```
Python
import os

cwd = os.getcwd()

print(f"Current Directory: {cwd}")
```

Expected Output

```
Current Directory: /home/user/robotics_project
```

Best Practice: Always use dynamic methods like `os.getcwd()` instead of hardcoded absolute paths, since hardcoded paths fail when files move to a different system.

2. Changing the Directory — `os.chdir()`

Changes the current working directory, affecting subsequent relative path lookups.

```
Python
import os

print(f"Starting in: {os.getcwd()}")
```



```
os.chdir('data')  
  
print(f"Now in: {os.getcwd()}")
```

Expected Output

Starting in: /home/user/project

Now in: /home/user/project/data

Best Practice: Prefer relative paths for project-internal folders for portability; reserve absolute paths for system-specific configuration.

3. Listing Directory Contents — `os.listdir()`

Returns a list of the names of entries in a given directory.

```
Python  
import os  
  
items = os.listdir('.')  
  
print("Directory contents:")  
  
for item in items:  
    print(f" - {item}")
```

Expected Output

Directory contents:

- main.py



- sensor_data.csv
- logs/

Note: `os.listdir()` returns both files and subdirectories, and does not search recursively. Use `os.path.isdir()` or `os.path.isfile()` to distinguish between them.

4. Creating Directories — `os.mkdir()` and `os.makedirs()`

-

Python

Single folder

```
import os
```

```
os.mkdir('new_folder')
```

Nested folders, created recursively

```
import os
```

```
os.makedirs('project/data/raw_sensors', exist_ok=True)
```

- `os.mkdir(path)` creates a single directory; it fails with `FileNotFoundError` if parent directories do not exist.
- `os.makedirs(path)` creates the target folder along with all necessary intermediate directories.
- Using `exist_ok=True` prevents a `FileExistsError` if the folders already exist.

5. Removing Directories — `os.rmdir()` and `os.removedirs()`

-



Python

```
import os

os.rmdir('empty_folder')

os.removedirs('project/data/raw_sensors')

print("Directories removed.")
```

- `os.rmdir(path)` removes a single, completely empty directory; raises `OSError` otherwise.
- `os.removedirs(path)` removes a directory and then its parent directories, stopping when a parent is not empty.

Warning: Both functions work only on empty directories, to prevent accidental data loss.

6. Renaming Files and Folders — `os.rename()`

Python

```
import os

old_name = 'temp_log.txt'

new_name = 'final_log_run1.txt'

if os.path.exists(old_name):

    os.rename(old_name, new_name)

    print(f"Renamed {old_name} to {new_name}")
```

Expected Output

Renamed temp_log.txt to final_log_run1.txt



Note: Checking existence before renaming prevents a `FileNotFoundError`. Renaming across directories on the same drive also works as a "move" operation.

7. Deleting Files — `os.remove()`

Python

```
import os

file_path = 'obsolete_data.csv'

if os.path.exists(file_path):
    os.remove(file_path)
    print(f"Deleted: {file_path}")
else:
    print("File not found.")
```

Warning: This deletes the file permanently, bypassing the Trash/Recycle Bin. `os.remove()` works on files only, not directories.

8. Checking Existence — `os.path.exists()`

Python

```
import os

config_file = 'robot_config.json'

if os.path.exists(config_file):
    print("Configuration found. Loading...")
```



```
else:  
    print("Configuration missing. Using defaults.")
```

Returns **True** if the path exists (file or directory), **False** otherwise. Checking existence before reading, renaming, or deleting is a key defensive programming practice.

9. Classifying Paths — **os.path.isfile()** and **os.path.isdir()**

```
Python  
import os  
  
target = 'sensor_data'  
  
if os.path.exists(target):  
    if os.path.isdir(target):  
        print(f"{target} is a directory.")  
    elif os.path.isfile(target):  
        print(f"{target} is a file.")  
else:  
    print(f"{target} does not exist.")
```

Expected Output

```
sensor_data is a directory.
```

Useful when iterating over **os.listdir()** results to separate files from folders.



10. Checking File Size — `os.path.getsize()`

Python

```
import os

file_path = "large_sensor_log.csv"

if os.path.exists(file_path):

    size_bytes = os.path.getsize(file_path)

    size_mb = size_bytes / (1024 * 1024)

    print(f"File size: {size_mb:.2f} MB")
```

Expected Output

File size: 14.50 MB

`os.path.getmtime()` can be used similarly to retrieve the last modification timestamp of a file.

Consolidated Functions Cheat Sheet

-

Function	Purpose
<code>os.getcwd()</code>	Get current working directory
<code>os.chdir(path)</code>	Change directory



Function	Purpose
<code>os.listdir(path)</code>	List contents of a directory
<code>os.mkdir(path)</code>	Create a single directory
<code>os.makedirs(path)</code>	Create recursive directories
<code>os.remove(path)</code>	Delete a file
<code>os.rmdir(path)</code>	Delete an empty directory
<code>os.rename(src, dst)</code>	Rename or move a file/directory
<code>os.path.exists(path)</code>	Check if path exists
<code>os.path.join(p1, p2)</code>	Safely concatenate paths

Practical Example 1: Robotics Project Folder Creator

-

Python

```
import os
```

```
def create_project_structure(project_name):
```



```
base_path = os.path.join(os.getcwd(), project_name)

folders = ['Code', 'Data', 'Images', 'Videos', 'Logs',
'Reports']

if not os.path.exists(base_path):
    os.mkdir(base_path)

    print(f"Created base folder: {project_name}")

    for folder in folders:
        f_path = os.path.join(base_path, folder)
        os.mkdir(f_path)
else:
    print("Project folder already exists!")

create_project_structure("Rover_Navigation_V1")
```

Key Points

- `os.path.join()` builds paths safely across Windows, macOS, and Linux.
- Iterating over a predefined folder list standardizes project structure.
- `os.path.exists()` prevents the script from crashing on repeated runs.



Practical Example 2: Automated Robot Log File Organizer

1.

Python

```
import os

import shutil # used to move files

def organize_logs():
    log_dir = "Logs"

    os.makedirs(log_dir, exist_ok=True)

    for item in os.listdir("."):
        if os.path.isfile(item) and (item.endswith(".txt") or
item.endswith(".csv")):
            src_path = item
            dst_path = os.path.join(log_dir, item)
            os.rename(src_path, dst_path) # moving file
            print(f"Moved {item} to {log_dir}/")

organize_logs()
```

Logic Breakdown

2. **Directory Scanning:** `os.listdir('.')` retrieves all items in the working directory.



3. **Targeted Filtering:** filters files by extension (`.txt`, `.csv`).
 4. **Safe Directory Creation:** `os.makedirs(..., exist_ok=True)` ensures the `Logs` folder exists.
 5. **File Relocation:** `os.rename()` moves matching files into the `Logs` folder.
-

Common Errors

FileNotFoundError

Occurs when a target file, or a required parent directory (for `os.mkdir()`), does not exist.

PermissionError

Occurs when the program attempts to access a protected system directory or a file owned by another user.

OSError — Directory Not Empty

Occurs when `os.rmdir()` is called on a folder that still contains files. The folder must be emptied first, or `shutil.rmtree()` used instead.

Incorrect Path Separators

Manually typing path separators (e.g., using `\` on Linux) breaks cross-platform execution; use `os.path.join()` instead.

Best Practices

- Always check whether a file or folder exists with `os.path.exists()` before operating on it.
- Avoid hardcoding absolute paths; use `os.getcwd()` and relative paths instead.
- Always construct paths with `os.path.join()` to handle path separators correctly across platforms.
- Wrap OS system calls in `try...except` blocks to gracefully handle permission or access errors.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



-
- Use descriptive, timestamped, or sequential file naming for automated logging.
-

Lab Task 1

Directory and Folder Automation

Objective

Write a single Python script using the `os` module to perform the following steps in order:

1. Print the current working directory to the console.
2. Create a new folder named `Robotics_Project_Lab`.
3. Inside it, create three subfolders: `src`, `data`, and `config`.
4. Rename the `config` folder to `settings`.
5. Delete the `data` folder (after confirming it is empty).

Execution Routine

1. Write a clean script using the `os` module for automation.
2. Run the script to automate the directory operations.
3. Verify the changes by checking your file explorer or by printing directory contents programmatically.

Verification Note: Always confirm your script worked by checking the file explorer after running it.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Lab Task 2

Robot Log Organizer

Objective

Extend the log organizer script so that it:

- Scans the current working directory for `.txt` and `.csv` files.
- Moves them into a `Logs` folder (creating it if it doesn't exist).
- Prints a confirmation message for each file moved.

Lab Task 3

Sensor Data Folder Creator

Objective

Write a script that creates a timestamped session folder (e.g., `run_20261027_1000`) for an autonomous rover, and inside it creates subfolders for `lidar`, `temperature`, and `logs`, using `os.makedirs()` with `exist_ok=True`.

Conclusion

In this lab, students learned how the Python `os` module bridges high-level program logic and the operating system to manage files, directories, and paths. They explored core functions for navigating, creating, renaming, and deleting files and folders, learned to check path existence and classify files versus directories, and applied these concepts to practical robotics automation tasks such as project



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



folder creation and log file organization. These concepts form the foundation for building reliable automation scripts and lead directly into file reading/writing and serial hardware communication.